Thread Test Assingment

Name : Umme kulsum (Suma)

ID : IT24052

```
import java.util.concurrent.atomic.AtomicLong;
public class Suma_Thread implements
Runnable {
static final AtomicLong safestaticCounter =
new AtomicLong (0);
static long Unsafe static counter = 0;
private long instanceCounter = 0;
private final long increments;
private final boolean threadsafe;
public Suma_Thread (long increments,
  boolean threadsafe) {
    this.increments = increments;
    this.threadsafe = threadsafe;
}
```

```
@ Override
public void run(){
 for (long (i=0; i< increments; i++){
  if (thread safe){
     safe staticCounter.incrementAndGet();
  } else {
    unsafestaticCounter ++;
  }
    instanceCounter ++;
  }
}
public static void main (string [] args)
 throws Interrupted Exception {
  if (args.length != 3){
System.out.println ("Usage :=java Suma_
Thread <thread> <increments> <true & false>);
 return;
}
```

```
int thread;
long increments;
boolean threadsafe;
try {
  threads = Integer.parseInt(args[0]);
  increments = Long.parseLong(args[1]);
  threadsafe = Boolean.parseBoolean(args[2]);
} catch (NumberFormatException e){
  System.out.print("Error: threads and
  increments must be integers");
    return;
  }
if (threads <1" increments < 0){
System.out.println("Error: threads must
  be    >= 1 and increments >= 0");
  return;
}
```

```
safe staticCounter.set(0);
Unsafe staticCounter = 0;
Suma_Thread[] tasks = new Suma_
Thread[threads];
Thread[] workers = new Thread[threads]
for (int i=0; i< threads; i++){
  tasks[i] = new Suma_Thread(
  increments, threadsafe);
    Workers[i] = new Thread(tasks[i],
    "Worker_" + (i+1));
  }
  for (Thread t: Workers) t.start();
  for (Thread t: Workers) t.join();
Long excepted = (long) threads *
  increments;
  long staticCount = threadsafe? safe
  staticCounter.get(); unsafe static
  Counter;
```

```
long nonStaticTotal = 0;
for (SumaThread t : tasks) nonStatic
Total += t.instanceCounter;
long absDiff = Math.abs (staticCount -
nonStaticTotal);
```

Compile and Run

```
javac SumaThread.java
java SumaThread 10 100000 Th
Mode : Thread-safe (AtomicLong)
Threads (N) : 10
Increments/thread (M) : 100000
Expected count : 1000000
Static count : 1000000
Non-static Total : 1000000
Absolute difference : 0
percentage Difference : 0.0000%
```

CamScanner